

AegisAI — Enterprise AI Security & Governance Gateway

Repository documentation and submission overview for Assignments 1–4.

Field	Value
Team ID / Group No.	7
Course	CS 543 — Web Services
Project	AegisAI — an enterprise control plane between an organisation's applications and AI model providers
Repository	https://github.com/neha-nupur/AegisAi
Written	8 October 2026

Team members

S. No.	Role	Name	Roll Number
1	Group Leader	Aditi Garg	20251651008
2	Group Member	Neha Nupur	20251651064
3	Group Member	Shivam Kumar Soni	20251651084

1. Links

GitHub repository (implementation + documentation)

<https://github.com/neha-nupur/AegisAi>

Public. Contains a readable, incremental commit history — one commit per task, not a single dump at the end.

Google Drive — working demonstration video

[PASTE GOOGLE DRIVE LINK HERE]

Action required before submission: record the demonstration, upload it to Google Drive, set sharing to “Anyone with the link” (Viewer), then replace the line above with the link. This placeholder is marked in red so it cannot be submitted unnoticed.

2. What this submission contains

AegisAI is developed across four assignments. Assignments 1 and 3 were completed for this submission; Assignments 2 and 4 were produced earlier and are already in the repository.

#	Assignment	Where	Submitted files
1	HTTP by Hand & Project Setup	repository root	README.md , docs/http-log.md , docs/network-analysis.md , brief.md
2	Services, Contracts and Schema	Assignment 2/	AegisAi.pdf , service diagram, ER diagram, AegisAiSchema.sql
3	Integrate an External SOAP Partner	Assignment 3/	integration.pdf , partner.wsdl , soap-request.xml , soap-response.xml , soap-fault.xml , verification.txt
4	Model & Provider Service in REST	Assignment 4/	openapi.yaml , implemented service with tests, resilience layer, curl-transcript.txt

Repository layout

```
AegisAi/
├── README.md           project documentation (A1 Task 3)
├── ReadMe.pdf          this document
├── brief.md            A1 Task 4 - what / who / nouns / verbs
├── docs/
│   ├── http-log.md     A1 Task 1 - six curl exchanges, one a 404
│   └── network-analysis.md A1 Task 2 - a real page's waterfall
├── Assignment 2/       design, diagrams, schema
├── Assignment 3/
│   ├── integration.pdf  A3 Tasks 1, 4, 5, 6
│   ├── partner.wsdl     A3 Task 2 - the partner contract
│   ├── soap-request.xml A3 Task 3 - request
│   ├── soap-response.xml A3 Task 3 - success response
│   ├── soap-fault.xml  A3 Task 3 - soap:Fault
│   └── verification.txt 27 machine checks over the four files above
└── Assignment 4/       OpenAPI, implementation, tests, evidence
```

3. Assignment 1 — HTTP by Hand & Project Setup

The assignment establishes the protocol foundation before any AegisAI code exists: read HTTP at the level of individual bytes, watch a production page load, and state the system precisely enough that the statements can later become an API contract.

Task 1 — docs/http-log.md

Six requests were issued by hand against the **GitHub REST API**, a public, read-only JSON API, using `curl -i` so the status line, every response header, the blank separator line and the body were all captured. Each pair records the exact command, the request as it went onto the wire, the full raw response, and a one-line note on what the status code and the `Content-Type` mean.

#	Request-target	Status	Content-Type	Meaning
1	GET /repos/neha-nupur/AegisAi	200	application/json	Success — repository representation returned in JSON.
2	GET /repos/neha-nupur/AegisAi/commits?per_page=1	200	application/json	Success — JSON array, pagination advertised in <code>link</code> .
3	GET /repos/neha-nupur/aegisai-does-not-exist	404	application/json	The deliberate failure — absent resource, reported as a typed JSON error.
4	GET /user	401	application/json	No credentials supplied for an identity-scoped resource.
5	GET /search/repositories?q=	422	application/json	Well-formed request, invalid semantics: <code>q</code> is empty.
6	GET http://api.github.com/zen	301	absent	Permanent redirect; <code>Content-Length: 0</code> , so nothing to type.

Task 2 — docs/network-analysis.md

A cold-cache load of this repository's own GitHub page, measured through the Resource Timing API and verified rather than assumed.

Question	Answer
Request count	174 — 1 document + 173 subresources
Total page size	2,094,889 bytes of encoded body ($\approx$ 2.00 MiB); 2,146,489 bytes on the wire
Slowest resource	<code>avatars.githubusercontent.com/u/110279038?s=60</code> — 75,003 ms and it never completed; the host times out from this network (<code>curl: (28) Connection timed out after 12006 milliseconds</code>), which held <code>window.load</code> to 76,074 ms
3xx / 4xx seen	3xx: none (<code>redirectCount = 0</code>). 4xx: one — <code>/_global-navigation/payloads.json</code> → 404 with a 52-byte typed body

Cache was genuinely disabled: **0 of 174 requests were served from cache**. As a control, the same page reloaded warm satisfied 168 of 176 subresources from cache and moved 6,334 bytes instead of 2,083,843 — a 99.7 % reduction that is exactly what the *Disable cache* checkbox exists to remove.

Task 3 — `README.md` and `docs/`

A real README describing the project and the repository structure, plus a readable history: each task was committed as it was finished rather than in one final commit.

Task 4 — `brief.md`

One page stating what AegisAI does and who uses it, then listing the vocabulary the API contract will be built from: **21 nouns** (organisation, AI request, policy, model, provider, approval, trace, ...) and **16 verbs** (authenticate, submit, scan, evaluate, mask, approve, route, retry, audit, ...), each with the contract it exposes.

4. Assignment 3 — Integrate an External SOAP Partner

Design-time only: no running server and no live gateway, and nothing here is a service AegisAI exposes. AegisAI is the *consumer*.

Task 1 — Context ([integration.pdf](#))

Partner: Meridian Messaging Ltd. — *Meridian SMS Notification Service*, v1. **Operation:** `sendSMS` .

AegisAI's approval workflow holds an AI request whenever policy returns `REQUIRE_APPROVAL` , and somebody has to be told. Meridian is the institute's SMS aggregator, it exposes SOAP 1.1 only, and that makes it AegisAI's single SOAP edge in an otherwise REST system — justified in four sentences on **strong contract** (the WSDL pins every field, type and cardinality, so a breaking change fails the build), **message-level security** (credentials, a freshness timestamp and an RSA signature travel inside the message and survive an intermediary terminating TLS) and **transactions** (a billable, non-idempotent side effect gets a typed `soap:Fault` instead of an arbitrary HTTP status to interpret).

Task 2 — [partner.wsdl](#)

A small but complete WSDL 1.1 contract carrying **all six mandatory elements**:

Element	Declared as
<code>wsdl:types</code>	Self-contained XSD — <code>E164</code> , <code>SenderId</code> , <code>Priority</code> , <code>DeliveryStatus</code> simple types plus <code>SendSMSRequest</code> , <code>SendSMSResponse</code> , <code>SendSMSFault</code>
<code>wsdl:message</code> ×3	<code>SendSMSInput</code> , <code>SendSMSOutput</code> , <code>SendSMSFaultMessage</code>
<code>wsdl:portType</code>	<code>NotificationPortType</code> , operation <code>sendSMS</code>
<code>wsdl:binding</code>	<code>NotificationSoap11Binding</code> — document/literal, SOAP 1.1 over HTTP, <code>soapAction</code> declared
<code>wsdl:service</code>	<code>NotificationService</code>
<code>wsdl:port</code>	<code>NotificationPort</code> → <code>https://api.meridian-sms.com/ws/v1/notifications</code>

Task 3 — the three envelopes

Each matches the WSDL element for element. The request puts its credentials in the Header and the call in the Body:

```
<soap:Header>
  <wsse:Security soap:mustUnderstand="1">
    <wsse:BinarySecurityToken ...>...</wsse:BinarySecurityToken>
    <wsu:Timestamp wsu:Id="ts"> ... </wsu:Timestamp>
    <wsse:UsernameToken wsu:Id="ut"> ... PasswordDigest ... </wsse:UsernameToken>
    <ds:Signature> ... SignedInfo / SignatureValue ... </ds:Signature>
  </wsse:Security>
</soap:Header>
<soap:Body wsu:Id="body">
  <sms:SendSMSRequest> ... </sms:SendSMSRequest>
</soap:Body>
```

The signature is real, not decorative. `verification.txt` re-parses the submitted file from disk and reports **27 checks passed, 0 failed**: the WSDL parses with all six elements and every internal reference resolving; the schema lifts cleanly out of `wsdl:types` ; **all three envelopes validate against that schema**; the request's child order equals the declared `xsd:sequence` ; both digests recompute from exclusive-C14N + SHA-1; the RSA-SHA1 signature verifies against the certificate embedded in the message itself; and the `PasswordDigest` recomputes as `Base64(SHA1(nonce + created + password))` .

Tasks 4, 5, 6 — `integration.pdf`

Task	Section	What it contains
4	HTTP binding	The POST as it goes on the wire — method, Host , Content-Type , the <code>SOAPAction</code> value from the binding and the endpoint URL from the <code>service/port</code> — with every line traced back to the declaration that produces it.
5	Discovery	WSDL-first with the contract pinned by SHA-256, plus a one-record catalogue entry carrying business, service, endpoint and a tModel-style pointer to the WSDL. No UDDI server is run.
6	Fault mapping	The partner's <code>faultCode</code> vocabulary mapped onto AegisAI's own error codes, an explicit list of strings that must never cross the boundary, and the audit record that preserves the diagnosability those strings provided.

Why the vocabulary must not leak. A caller that learns `CARRIER_REJECTED` is depending on a vendor it has no contract with. Swapping Meridian for another aggregator would then break callers AegisAI never told about Meridian. The gateway returns only `notification_unavailable` , `503` and a `correlationId` ; the partner's words go into the audit record, keyed by that same correlation ID, where an operator can still find them.

5. How to verify this submission

1. **Read the history.** `git log --oneline` shows one commit per task, in the order the work was done.
2. **Reproduce Assignment 1.** Run the six `curl -i` commands exactly as printed in `docs/http-log.md`; they are read-only and need no credentials.
3. **Reproduce the waterfall.** Open the repository on GitHub, DevTools → Network, tick *Disable cache*, reload, and compare the request count and transferred bytes against `docs/network-analysis.md`.
4. **Re-run the SOAP checks.** The four files in `Assignment 3/` are well-formed and mutually consistent; `verification.txt` records the 27 checks and how each was made.
5. **Watch the demo.** The Google Drive link in section 1.

Before submitting: replace the `[ PASTE GOOGLE DRIVE LINK HERE ]` placeholder in section 1 with the real Drive URL, shared as “*Anyone with the link*”, and confirm the GitHub link above resolves.

CS 543 — Web Services · Submission ReadMe · Team ID 7

Aditi Garg (20251651008) · Neha Nupur (20251651064) · Shivam Kumar Soni (20251651084)

AegisAI — Enterprise AI Security & Governance Gateway · <https://github.com/neha-nupur/AegisAi>